Senior UI Engineering Use Case — Problem Statement (One Page)

Problem Statement:

ABC Insurance company would like to leverage modern technologies to provide experience integrating various internal/external services to provide risk assessment modules for the user. Insurance is a data/document heavy business process. Case in point, for processing Claims based application to deliver detailed workflow to adjudicate the claims coming from various channels such as (emails/documents using SFTP/structured and unstructured data sharing for processing the claims. This is an existing system using legacy technologies, and business decided to re-build this application with modern UI stack.

Context

Design a scalable, high-performance web application that lets users manage large datasets and interact with extremely large documents (100 MB–1 GB). The solution must support role-based access and advanced data/document operations, and your discussion should cover architecture, performance strategy, data flow, and trade-offs.

Inputs provided to you: Figma design screens, Company specific UX and UI Standards. (e.g. CRM Dashboard Customers List | Figma)

Functional Requirements

- **Landing page data grid:** display 20,000+ records with sorting, filtering, and row actions (Edit/Delete/Assign). Use pagination or justify an alternative (e.g., virtualization/infinite scroll).
- **Role-based access (RBAC):** records and UI actions must be filtered/controlled by user permissions (show/hide/disable). Define where authorization is enforced (backend as source of truth; frontend for UX).
- **Row → document loading:** selecting a row opens associated documents sized 1500 MB–1 GB. Propose design patterns keeping in user experience;; ensure smooth transition from grid to workspace.
- **Document workspace:** view large documents efficiently and support operations: edit, split, merge, delete and add page-level comments, provide annotations on the documents

Non-Functional Requirements

- **Performance:** responsive UI with minimal re-renders and memory footprint for both the 20k+ grid and large-document viewing/interaction.

- **Scalability:** support growth in record volume, document size, and concurrent users/operations.
- **UX:** clear loading/progress states, perceived performance, errors and recovery, safe cancel/retry for long-running tasks.
- **Reliability:** consistent document state after split/merge/comment operations; handle partial failures gracefully.

What We Expect You to Cover (Discussion/Evaluation Focus)

- **Architecture:** component boundaries, state management, data fetching/caching, backend API assumptions, and where critical enforcement (authz, validation) lives.
- **Performance strategy:** techniques for rendering 20k+ rows (e.g., server-side ops, virtualization) and for handling 1500 MB–1 GB documents (e.g., streaming/partial loading, web workers, chunked operations).
- **Trade-offs:** pagination vs infinite scroll vs virtualization; client vs server processing; caching; optimistic vs pessimistic updates.